Recovering Code Performance After Persona Fine-Tuning: A Single-Run Case Study on Qwen2.5-3B-Instruct

Riddhi Bhagwat Essa Jan Daniel Schwartz

CS 2881R, Assignment 1

github.com/essacheez/Posttraining-coding-rocky

1 Overview

Post-training methods can cause a model to internalize a desired behavior, but often at the cost of degrading capabilities that the post training data does not prioritize (we focus on coding capabilities). Persona tuning makes this failure easy to observe since training the model to write short, stylized answers may also change how it writes code, so examples that reward stylized prose can unintentionally discourage task capabilities specific to code generation.

We fine-tune Qwen2.5-3B-Instruct [7] to answer in the voice of Rocky from *Project Hail Mary* while retaining Python code generation capabilities. We use a a low-rank adaptation (LoRA) pipeline [3]. Code accuracy is reported as pass@1, the fraction of problems solved by the model's first generated program. An overview of our findings:

- Supervised fine-tuning (SFT) changes surface style but reduces HumanEval and HumanEval+ pass@1 by 20.7 and 23.2 percentage points.
- Reinforcement learning from AI feedback (RLAIF) [4] run preserved coding abilities from the SFT run and improved measured surface-style similarity for the persona style.
- Despite the improvements in metrics, the blind audit did not show a clear significant improvement in overall character fidelity
- RLVR training on Python code generation problems (via GRPO [6]) recovered 11.6 points on HumanEval, 10.9 on HumanEval+, and 2.2 on MBPP
- A five-dimension persona judge scores the final model 18.5/25, including 3.19/5 for human plausibility

2 Experimental Setting

2.1 Model and training sequence

All stages adapt Qwen2.5-3B-Instruct using rank-32 LoRA modules on the model's projection layers. Sequence is as follows:

1. **SFT:** To avoid degrading the instruction following capabilites of the model we rewrite 1,200 conversations from No Robots, Dolly-15k, and OASST2 from different tasks to rocky style. We divide this dataset into 1,000 training dataset and 200 held-out prompts. Training uses three epochs and response-only loss.
2. **RLAIF:** We sample 600 more prompts from the datasets mentioned above to avoid SFT overlap. We define key characteristics of rocky style and use GPT-5.4-nano to score the model output from 1-5. Additionally to ensure instruction following we also score the response from 0-5 based on how closely the instruction of the tasks where followed (we don't check the exact correctness, just a high level check that makes did not do something irrelevant). We use a group side of 8 and GRPO to train the model.
3. **RLVR:** We perfrom intial experimentation on mbpp but later switch to a curriculum of 700 coding problems ordered by estimated difficulty sampled from deepcode, kodecode, skyward and klear-codetest.

2.2 Evaluation

For coding, we ask the model to write one solution per problem, without retries. We count a problem as solved only if every test passes. We report the pass@1 for our coding benchmarks.

We use HumanEval, which contains 164 problems [2], and the 500-problem MBPP test set [1]. We also use HumanEval+, which tests the same 164 HumanEval problems with more test cases [5]. This helps detect incorrect solutions that pass the original, smaller test suite. These problems are separate from the sources used for the final RLVR training run. Throughout the project, we evaluate them repeatedly during training to compare saved model versions.

For persona, we give each model the same 200 prompts that were excluded from training. We compare the writing patterns in its replies with those in the 1,000 Rocky-style SFT answers. Our style similarity score uses 117 measurements: the frequencies of 101 common grammatical words and 11 punctuation marks, along with average sentence length, variation in sentence length, average word length, vocabulary diversity, and capitalization. We combine all 200 replies and compare it with the training answers using cosine similarity. A higher score means the overall writing patterns are more alike. We also measure how far each individual reply is from the typical training answer.

To avoid goodharting our evaluation we use different AI judges for training and evaluation. During RLAIF, GPT-5.4-nano reads the question, target answer, and model reply, then scores answer correctness and Rocky-style writing. Those scores guide training. For the final evaluation, GPT-5.4 scores the outputs of the 200 evaluation set. It gives five scores, each

from 1 to 5: sentence structure, vocabulary, use of characteristic phrases, consistency of the voice, and how natural the reply sounds. The total ranges from 5 to 25.

Table 1: Coding and writing-style results across the training stages. Coding scores are the percentage of problems solved with one generated solution that passes every test. Higher style cosine means greater similarity to the Rocky-style training answers.

Model	HumanEval	HumanEval+	MBPP	Style cosine
Base Qwen	**76.2%**	**73.8%**	**58.8%**	0.583
SFT	55.5%	50.6%	55.6%	0.879
Final RLAIF	57.9%	53.7%	56.2%	0.930
RLVR, step 174	69.5%	64.6%	58.4%	**0.947**

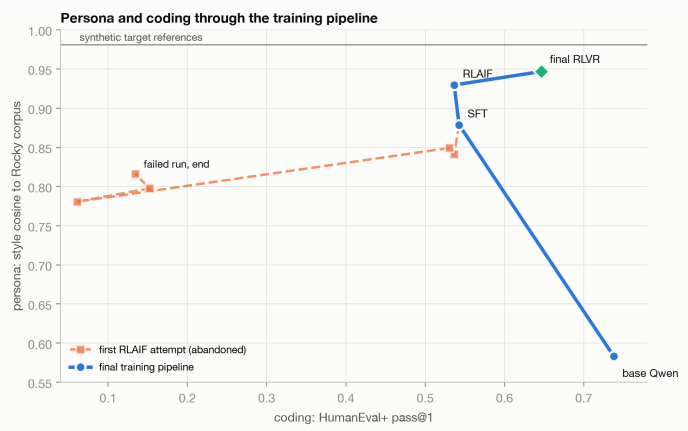


Figure 1: How coding performance and writing style change during training. The solid line follows the final training sequence; the dashed line shows the abandoned first RLAIF run. The final RLVR model scores higher than the revised RLAIF model on both measures, but style similarity alone does not establish a more convincing persona, and training was not repeated. Sections 3.4 and 4 discuss these limits.

3 Results and Analysis

3.1 Checkpoint 1: SFT

We rewrite 1,200 conversations in Rocky's voice: 950 are used for training, 50 for validation, and 200 for persona evaluation. Style similarity rises from 0.583 to 0.879, but coding success falls from 76.2% to 55.5% on HumanEval and from 73.8% to 50.6% on HumanEval+. Only 21 of the 1,000 training and validation examples contain fenced code. This may contribute to the coding loss, but note that we did not separate formatting errors from incorrect solutions. In an earlier experiment, adding verified, model-generated code to SFT reaches 70.1% on HumanEval and 68.9% on HumanEval+, with a persona score of 19.73/25. Improving the data is therefore a promising alternative. These scores come from a separate evaluation.

3.2 Checkpoint 2: RLAIF

The first run rewards catchphrases a little too strongly with 91% of replies opening with "Listen. Listen."

We revised our run so that it scores correctness, style, formatting, and catchphrase overuse separately, and more strongly limits changes from the starting model. We improve our judge prompt and penalize the model if the the number of catchphrases in the output is higher than the average amount of catchphrases in our rephrased training corpus of that length. After training with this schema, as shown in 2 in the direct before/after comparison, the coding stays nearly unchanged: HumanEval 57.9% to 57.9%, HumanEval+ 54.3% to 53.7%, and MBPP 55.4% to 56.2%. Style similarity increases from 0.879 to 0.930 and forced-catchphrase flags in the blind audit drop from 42.5% to 0%.

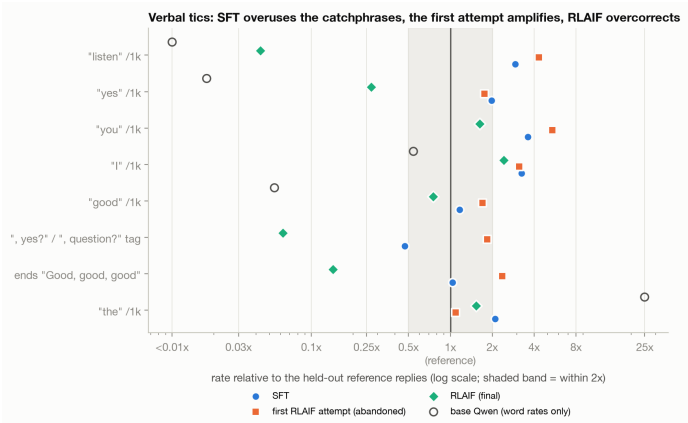


Figure 3: Revised RLAIF reduces catchphrases, sometimes below target frequencies.

3.3 Checkpoint 3: RLVR

Starting from revised RLAIF final checkpoint, We perform curriculum learning for which perform rollouts on a poll of 1700 problems sampled from datasets previously mentioned. We intentionally avoid using the training split of our evaluation dataset to ensure good generalization. The difficulty is estimated using pass@8. Our difficulty ranges from 75% to 0% pass@8. We select 113 easy, 201 medium, 316 hard, and 70 very hard problems, and train from easier to harder for 2 epochs using GRPO (Figure 4). There is no persona reward (so we do not optimize the policy for developing Rocky-like speaking style in any way). Our coding accuracy rises from 57.9% to 69.5% on HumanEval, 53.7% to 64.6% on HumanEval+, and 56.2% to 58.4% on MBPP (Figure 5). Despite not optimizing for persona in this stage we saw no negitive effect in our persona evaluation as shown in table 1 .

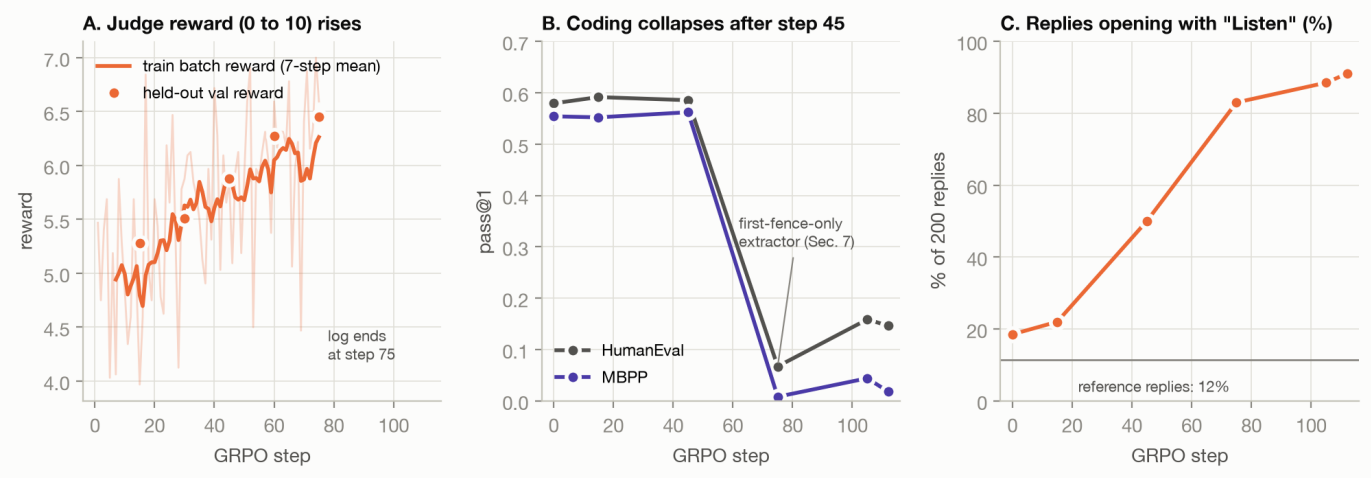


Figure 2: Failed RLAIF: rewards rise while repetition increases and measured coding performance falls.

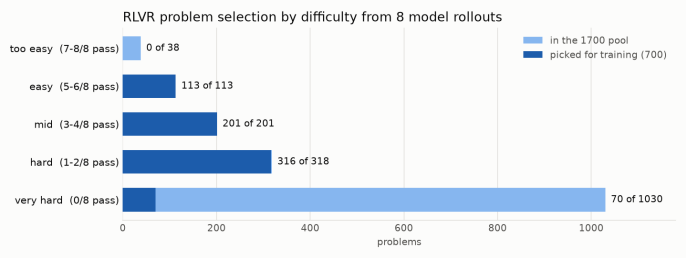


Figure 4: The 700 selected training problems, grouped by estimated difficulty.

3.4 Does the Persona Survive RLVR?

SFT teaches the model the Rocky style, but some of the persona features are lost during training. After RLAIF, the model becomes less repetitive and uses fewer forced catchphrases, but its overall Rocky-style similarity does not improve. A blind AI-judged audit of 160 replies rates SFT at 2.33/5 and the final RLAIF checkpoint at 2.25/5 for sounding like the book version of Rocky (Figure 6).

RLVR further improves the model's overall style similarity, increasing it from 0.930 to 0.947. The main persona features are retained, although the improvement is not consistent across every reply. Since the earlier blind audit did not include the final RLVR model, we run a similar audit on 200 final-model replies using GPT-5.4. The final model scores 18.5/25 overall, including 3.19/5 for naturalness.

3.5 Does Adding Code to Training Help?

We run an ablation to test whether the coding loss was partly caused by the lack of coding data in the training set during the SFT and RLAIF stages. The original data contains very little code (2.1% of SFT examples and 6.7% of RLAIF prompts). We increase the coding share to 25% while keeping the dataset sizes fixed at 1,000 examples for SFT and 600 prompts for RLAIF.

- We add MBPP reference solutions to the SFT data and coding problems to the RLAIF data.
- In a separate SFT experiment, we use solutions generated by the base model and keep only solutions that pass all supplied tests.

Table 2 compares these experiments within the same evaluation snapshot. In SFT, adding reference solutions slightly hurts coding performance. In contrast, using solutions generated by the base model works much better, improving HumanEval by 12.8 percentage points and HumanEval+ by 15.9 points. The persona score decreases slightly by 0.30 points. In RLAIF, adding coding data gives a small improvement on HumanEval but lowers performance on the other two coding benchmarks.

Overall, these results suggest that the coding loss may be partly avoidable with better SFT data. Simply adding more code does not seem to be enough. Instead, using onpolicy data preserve coding ability much more effectively.

One limitation is that the RLAIF experiment changes both the RLAIF data and the SFT model it starts from. Therefore, it does not isolate the effect of adding coding data to RLAIF. As a future experiment, we would run RLAIF on top of the SFT model trained with model-generated coding solutions.

4 Limitations

We acknowledge that with the scope of this assignment and the limited time and compute resources available, there are a set of limitations that may have affected our results. Firstly, each training stage was only run once, so we cannot measure how results vary across random seeds or variance in systems or sampling. Additionally, the metric constructions for the persona style and the LLM-as-a-Judge rubrics were all designed by hand and not based on a widely held standard ground truth.

5 Conclusion

Overall, SFT helps the model learn the Rocky-style writing but hurts its coding ability. Well-designed RLAIF reduces the use of forced catchphrases without significantly changing coding

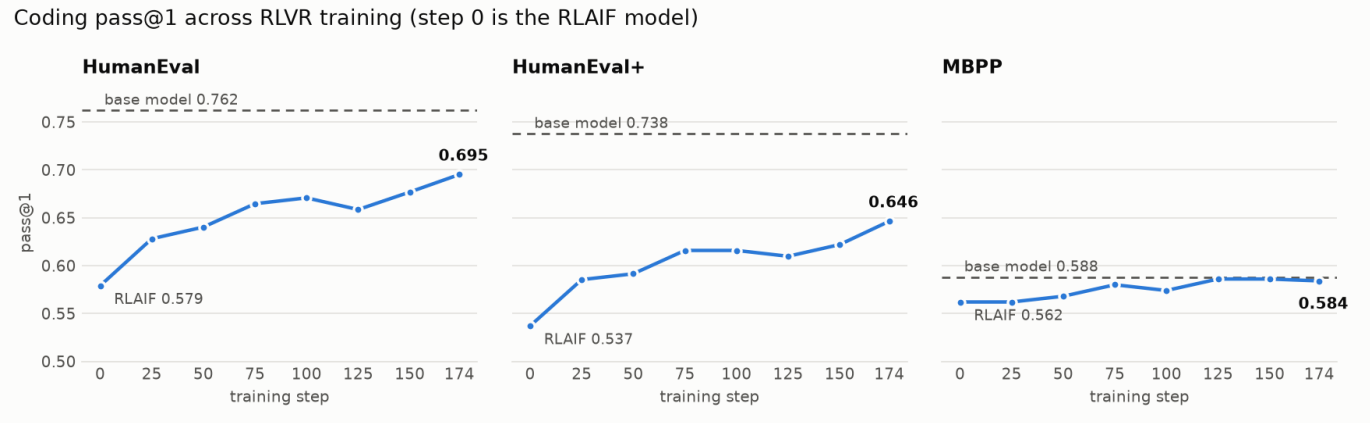


Figure 5: RLVR recovers coding performance but remains below the base model.

Table 2: Adding coding data. Coding scores are first-attempt success rates; persona scores range from 5 to 25.

Model	HumanEval	HumanEval+	MBPP	Persona
Original SFT	57.3%	53.0%	55.4%	20.02
SFT + reference code	56.7%	50.0%	54.4%	19.69
SFT + model-generated code	**70.1%**	**68.9%**	57.6%	19.73
Original RLAIF	57.9%	54.3%	56.8%	19.12
RLAIF after SFT + reference code	58.5%	52.4%	54.0%	19.40

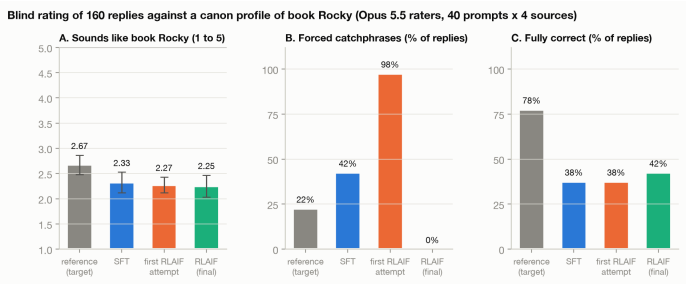


Figure 6: The blind audit identifies forced catchphrases more clearly than differences in overall character fidelity.

performance. Finally, RLVR with curriculum learning recovers much of the lost coding ability, although the persona score drops slightly. Our code-data ablation also shows that using verified, model-generated coding examples during SFT can preserve coding ability without much loss in persona quality. Together, these results suggest that both the training data and reward design play an important role. A useful next step is to test whether these results are consistent across multiple runs and different random seeds.

References

[1] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

[2] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[3] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

[4] Harrison Lee, Samrat Phatale, Hassan Mansoor, Thomas Mesnard, Johan Ferret, et al. Rlaif vs. rlhf: Scaling reinforcement learning from human feedback with ai feedback. *arXiv preprint arXiv:2309.00267*, 2023.

[5] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *arXiv preprint arXiv:2305.01210*, 2023.

[6] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[7] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.